Capítulo 5

Programação Orientada a Objetos (POO)

Exercícios

Para fixar os conceitos apresentados, propomos a seguir uma série de exercícios.

1. (TEORICA): Encapsulamento e acesso a dados: Considere que um desenvolvedor criou uma classe Pessoa com um atributo __idade. Sobre o acesso e uso desse atributo, qual das afirmações é **correta**?

A. Como __idade é privado, ele não pode ser alterado nem mesmo por métodos dentro da classe.
B. O atributo __idade pode ser lido e modificado livremente fora da classe, já que Python não possui encapsulamento real.
C. O atributo __idade só deve ser acessado através de métodos da classe (por exemplo, um getter e/ou setter), pois não é acessível diretamente de fora devido ao encapsulamento implementado pelo nome.
D. Em Python, __idade e _idade (um underscore) têm o mesmo efeito – ambos são públicos.

2. (TEÓRICA): Pilares da POO: A seguinte situação é descrita: *"Uma classe base Veiculo define um método mover(). Duas subclasses, Carro e Barco, implementam mover() de formas diferentes. Em uma parte do programa, uma função recebe um objeto Veiculo e chama obj.mover(), sem saber se é Carro ou Barco – ainda assim, o comportamento correto acontece."* Essa situação ilustra principalmente qual pilar da orientação a objetos?

A. Abstração
B. Encapsulamento
C. Herança
D. Polimorfismo

3. (TEÓRICA): Comparando paradigmas: Qual das opções **melhor diferencia** a programação procedural da programação orientada a objetos?

A. Na programação procedural, usa-se funções; na orientada a objetos, não existem funções.
B. A programação orientada a objetos organiza o código em classes/objetos que combinam estado e comportamento, enquanto a procedural organiza o código em rotinas (funções) separadas dos dados.
C. A programação procedural é sempre menos eficiente que a orientada a objetos.
D. A orientação a objetos não permite estrutura de controle como loops e condicionais, ao contrário do procedural.

4. (TEÓRICA) Conceitos de Python: Em relação à POO em Python, assinale a alternativa **incorreta**:

A. É possível criar atributos que sejam exclusivos da classe (estáticos), compartilhados entre todas as instâncias.
B. O método __str__ de uma classe, quando implementado, define o que print(obj) exibirá para objetos daquela classe.
C. A herança em Python não suporta mais de uma classe base; cada classe só pode herdar de no máximo uma superclasse.
D. Mesmo sem usar modificadores de acesso explícitos, é prática comum usar convenções como _atributo e __atributo para indicar níveis de acesso (semiprivado/privado).

5. (PRÁTICA): *Classe Pessoa*: Crie uma classe Pessoa com nome e idade. Adicione método faz_aniversario() que incrementa a idade em 1. Use __str__() para retornar algo como "<nome>, <idade> anos". Crie duas pessoas e mostre a idade de cada uma antes e depois de chamar faz_aniversario().

6. (PRÁTICA): *Encapsulamento*: Dada a classe abaixo sem encapsulamento, implemente encapsulamento adequado:

```
class Lampada:
    def __init__(self):
        self.estado = False

    def ligar(self):
        self.estado = True

    def desligar(self):
        self.estado = False
```

Modifique para que o atributo estado seja privado e só possa ser alterado pelos métodos `ligar()` e `desligar()`. Adicione métodos `is_acesa()` para verificar o estado.

7. (PRÁTICA): *Herança*: Implemente duas classes com herança:

- Classe base Animal com um método fazer_som() que imprime algo genérico, por exemplo "Algum som".
- Duas subclasses Cachorro e Gato que herdam de Animal. Na classe Cachorro, sobrescreva fazer_som() para imprimir "Au Au". Em Gato, para "Miau". Crie um objeto de cada e chame fazer_som() em cada um.

8. (PRÁTICA): *Polimorfismo*: Dada as classes do Exercício 4, escreva uma função externa animais_falam(lista_animais) que recebe uma lista de objetos (que podem ser Cachorro ou Gato) e chama fazer_som() em cada um. Mostre que a função pode operar sobre diferentes tipos de animais (polimorfismo em ação).

Projetos

Projeto 1: Biblioteca de Livros

Contexto do mundo real: Imagine um sistema para gerenciar o empréstimo de livros em uma pequena biblioteca escolar. Queremos cadastrar livros e usuários, registrar quando alguém empresta ou devolve um livro.

Descrição do problema: Cada **Livro** tem título, autor, quantidade disponível. Cada **Usuário** tem nome, matrícula e lista de livros emprestados. O sistema deve permitir que um usuário faça empréstimos (reduzindo a quantidade disponível) e devoluções (aumentando quantidade). Devemos evitar empréstimos se o livro estiver indisponível. Também precisamos manter os dados organizados e seguros (ex.: não permitir mexer diretamente na quantidade disponível).

Projeto 2: Gerenciamento de Conta Bancária com Herança

Contexto do mundo real: Um banco que oferece dois tipos de conta: conta corrente e conta poupança. Queremos um sistema que permita depositar, sacar e aplicar juros (no caso da poupança).

Descrição do problema: Precisamos modelar contas bancárias em Python de forma orientada a objetos. Teremos:

- Classe base Conta com atributos comuns (titular, saldo) e métodos depositar(valor) e sacar(valor).
- Classe ContaCorrente que herda de Conta e adiciona, por exemplo, taxa de saque (ou limite especial).
- Classe ContaPoupanca que herda de Conta e tem um método extra render_juros() que aumenta o saldo de acordo com uma taxa fixa anual.
  Também podemos manter atributos de classe (como juros = 0.05 para 5% ao ano).